



Tecnológico de Monterrey

Escuela de Ingeniería y Ciencias

Monterrey Campus

Autonomy of Unmanned Aerial Vehicles

MR3003C.601

Report:

Practice 09

State Feedback Gain Matrix for Rotary Inverted Pendulum

Part A. State Space Modelling

Name:

Carlos Hernán Auquilla Larriva | A00834778

Teachers:

Carlos Gustavo Sotelo Molina, Ph.D

David Alejandro Sotelo Molina, Ph.D

Due date:

October 2nd, 2024

Table of contents

1. Linear State Space Model of the Pendulum System	3
2. Matlab Code with Linear State Space Model	5
Matlab Script.....	5
Computed Values for the State Space Matrices.....	6
3. Response of the Rotary Pendulum System for the Virtual Lab and the Model (Simulink)	7
Simulink Block Diagram	7
Figures.....	8
4. Linear State Response of the Rotary Pendulum System for the Virtual Lab and the Model (Matlab).....	10
Matlab Script to plot figures	11
5. Conclusion	12

1. Linear State Space Model of the Pendulum System

$\theta(t) \rightarrow \text{Arm Angle}$ $\alpha(t) \rightarrow \text{pendulum angle}$ + CCW

$$x_1(t) = \theta(t) \quad \dot{x}_1(t) = \dot{\theta}(t) = x_3(t) \quad \text{eq.1}$$

$$x_2(t) = \alpha(t) \quad \dot{x}_2(t) = \dot{\alpha}(t) = x_4(t) \quad \text{eq.2}$$

$$x_3(t) = \dot{\theta}(t) \quad \dot{x}_3(t) = \ddot{\theta}(t)$$

$$x_4(t) = \dot{\alpha}(t) \quad \dot{x}_4(t) = \ddot{\alpha}(t)$$

$$\ddot{\theta}(t) = \frac{1}{J_t} [m_p l^2 g \alpha(t) - J_p b_r \dot{\theta}(t) + m_p l r b_p \dot{\alpha}(t) + J_p \tau]$$

$$\dot{x}_3(t) = \frac{1}{J_t} \left\{ m_p l^2 g x_2(t) - J_p b_r x_3(t) + m_p l r b_p x_4(t) + J_p \left[\frac{k_m}{R_m} [V_m(t) - k_m \dot{\theta}(t)] \right] \right\}$$

$$\dot{x}_3(t) = \frac{1}{J_t} \left\{ m_p l^2 g x_2(t) - J_p b_r x_3(t) + m_p l r b_p x_4(t) + J_p \left[\frac{k_m}{R_m} [u(t) - k_m x_3(t)] \right] \right\}$$

$$\dot{x}_3(t) = \frac{1}{J_t} \left\{ m_p l^2 g x_2(t) - J_p \left[b_r + \frac{k_m^2}{R_m} \right] x_3(t) + m_p l r b_p x_4(t) + J_p \left[\frac{k_m}{R_m} u(t) \right] \right\} \quad \text{eq.3}$$

$$\ddot{\alpha}(t) = \frac{1}{J_t} [-m_p g l J_r \alpha(t) + m_p l r b_r \dot{\theta}(t) - J_p b_p \dot{\alpha}(t) - m_p r l J_r]$$

$$\dot{x}_4(t) = \frac{1}{J_t} \left\{ -m_p g l J_r x_2(t) + m_p l r b_r x_3(t) - J_p b_p x_4(t) - m_p r l \left[\frac{k_m}{R_m} [V_m(t) - k_m \dot{\theta}(t)] \right] \right\}$$

$$\dot{x}_4(t) = \frac{1}{J_t} \left\{ -m_p g l J_r x_2(t) + m_p l r b_r x_3(t) - J_p b_p x_4(t) - m_p r l \left[\frac{k_m}{R_m} [u(t) - k_m x_3(t)] \right] \right\}$$

$$\dot{x}_4(t) = \frac{1}{J_t} \left\{ -m_p g l J_r x_2(t) + m_p l r \left[b_r + \frac{k_m^2}{R_m} \right] x_3(t) - J_p b_p x_4(t) - m_p r l \left[\frac{k_m}{R_m} u(t) \right] \right\} \quad \text{eq.4}$$

From eq. 1, eq. 2, eq. 3, eq. 4

$$\begin{bmatrix} \dot{x}_1(t) \\ \dot{x}_2(t) \\ \dot{x}_3(t) \\ \dot{x}_4(t) \end{bmatrix} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & \frac{m_r^2 l^2 r g}{J_t} & \frac{-J_p}{J_t} \left(b_r + \frac{k_m^2}{R_m} \right) & \frac{m_p l r b_p}{J_t} \\ 0 & \frac{-m_p g l J_r}{J_t} & \frac{m_p l r}{J_t} \left(b_r + \frac{k_m^2}{R_m} \right) & \frac{-J_p b_p}{J_t} \end{bmatrix} \begin{bmatrix} x_1(t) \\ x_2(t) \\ x_3(t) \\ x_4(t) \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ \frac{k_m J_p}{R_m J_t} \\ \frac{-m_p r l k_m}{R_m J_t} \end{bmatrix} u(t)$$

$$\begin{bmatrix} y_1(t) \\ y_2(t) \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} x_1(t) \\ x_2(t) \\ x_3(t) \\ x_4(t) \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \end{bmatrix} u(t)$$

Alternate form

$$A = \left[\frac{1}{J_t} \right] \begin{bmatrix} 0 & 0 & J_t & 0 \\ 0 & 0 & 0 & J_t \\ 0 & m_r^2 l^2 r g & -J_p \left(b_r + \frac{k_m^2}{R_m} \right) & m_p l r b_p \\ 0 & -m_p g l J_r & m_p l r \left(b_r + \frac{k_m^2}{R_m} \right) & -J_p b_p \end{bmatrix}$$

2. Matlab Code with Linear State Space Model

Matlab Script

```
A =  
      x1      x2      x3      x4  
x1      0      0      1      0  
x2      0      0      0      1  
x3      0     152    -12.53    0.5005  
x4      0   -264.3     12.39   -0.5064  
  
B =  
      u1  
x1      0  
x2      0  
x3     56.98  
x4    -56.32  
  
C =  
      x1  x2  x3  x4  
y1      1   0   0   0  
y2      0   1   0   0  
  
D =  
      u1  
y1      0  
y2      0  
  
% Resistance  
Rm = 7.5;  
% Current-torque (N-m/A)  
kt = 0.0422;  
% Back-emf constant (V-s/rad)  
km = 0.0422;  
%  
% Rotary Arm  
% Mass (kg)  
mr = 0.095;  
% Total length (m)  
r = 0.085;  
% Moment of inertia about pivot (kg-m^2)  
Jr = 2.2879e-4;  
% Equivalent Viscous Damping Coefficient (N-m-s/rad)  
br = 1e-3; % damping tuned heuristically to match QUBE-Sero 2 response
```

```

%
% Pendulum Link
% Mass (kg)
mp = 0.024;
% Total length (m)
Lp = 0.129;
% Pendulum center of mass (m)
l = 0.0645;
% Moment of inertia about pivot (kg-m^2)
Jp = 1.3313e-4;
% Equivalent Viscous Damping Coefficient (N-m-s/rad)
bp = 5e-5; % damping tuned heuristically to match QUBE-Sero 2 response
% Gravity Constant
g = 9.81;

% Find Total Inertia
Jt = Jp*Jr-mp^2*l^2*r^2;

%% Step 2: Program and compute the State Space Representation matrixes
A = (1/Jt)*[0 0 Jt 0;
            0 0 0 Jt;
            0 mp^2*l^2*r*g -Jp*(br+km^2/Rm) mp*l*r*bp;
            0 -mp*g*l*Jr mp*l*r*(br+km^2/Rm) -Jp*bp];

B = [0
      0;
      km*Jp/(Rm*Jt);
      -mp*r*l*km/(Rm*Jt)];

C = [1 0 0 0;
      0 1 0 0];

D = [0;0];

sys = ss(A,B,C,D);

```

Computed Values for the State Space Matrices

```

A =
      x1      x2      x3      x4
x1      0      0      1      0
x2      0      0      0      1
x3      0     152    -12.53    0.5005
x4      0   -264.3     12.39   -0.5064

B =
      u1
x1      0
x2      0
x3     56.98
x4    -56.32

C =
      x1      x2      x3      x4
y1      1      0      0      0
y2      0      1      0      0

D =
      u1
y1      0
y2      0

```

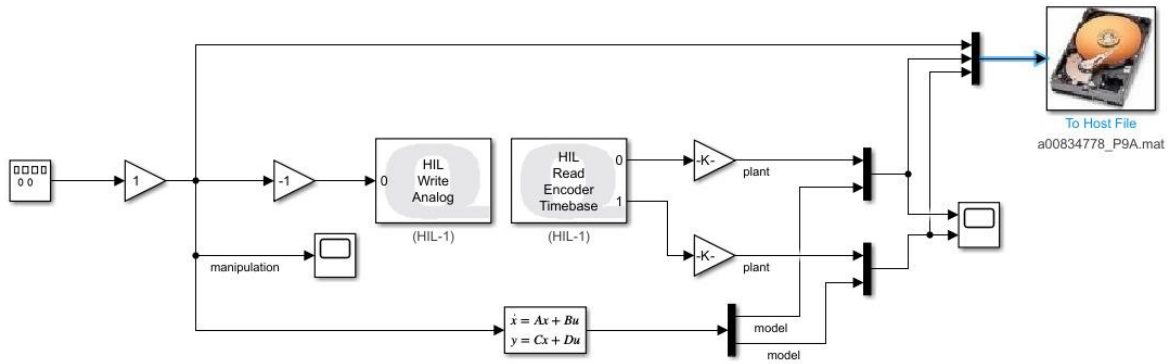
3. Response of the Rotary Pendulum System for the Virtual Lab and the Model

(Simulink)

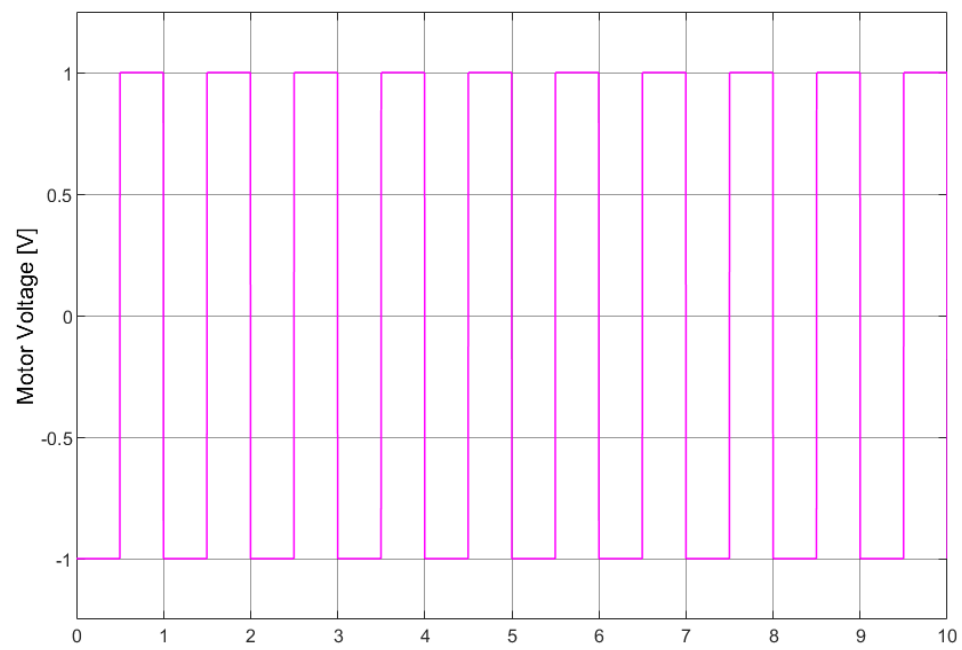
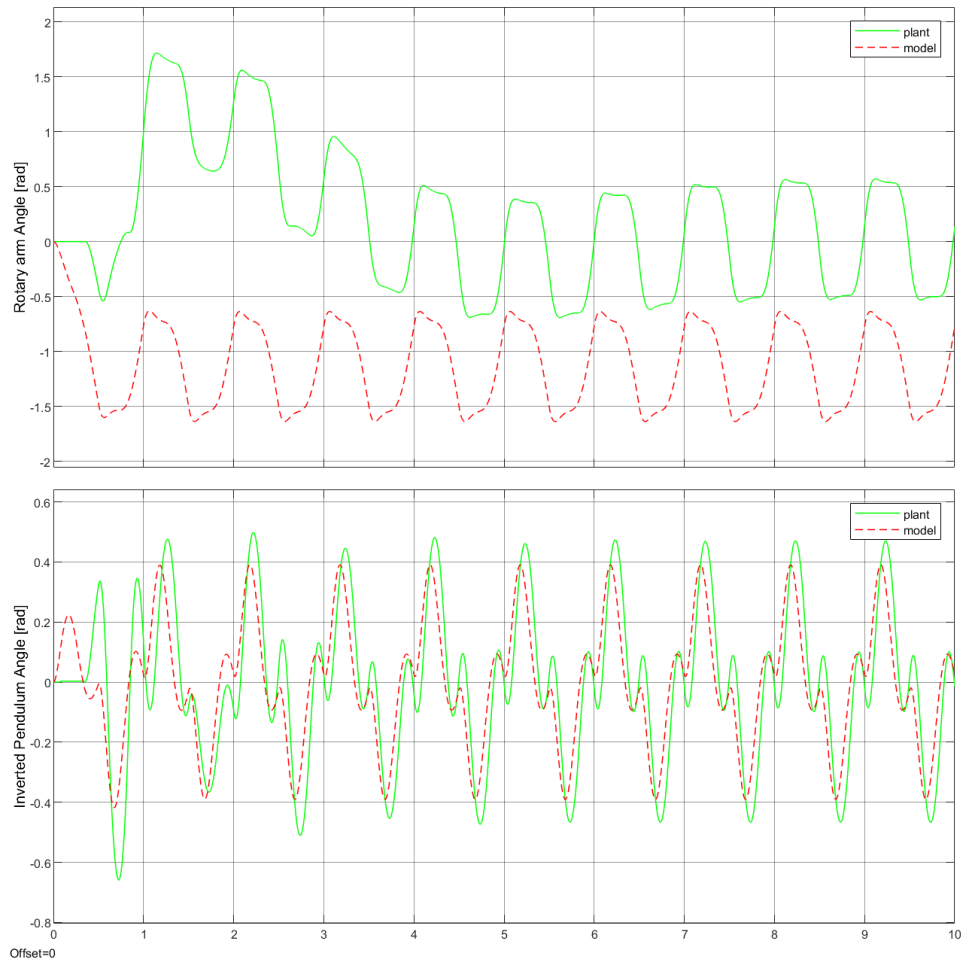
Simulink Block Diagram



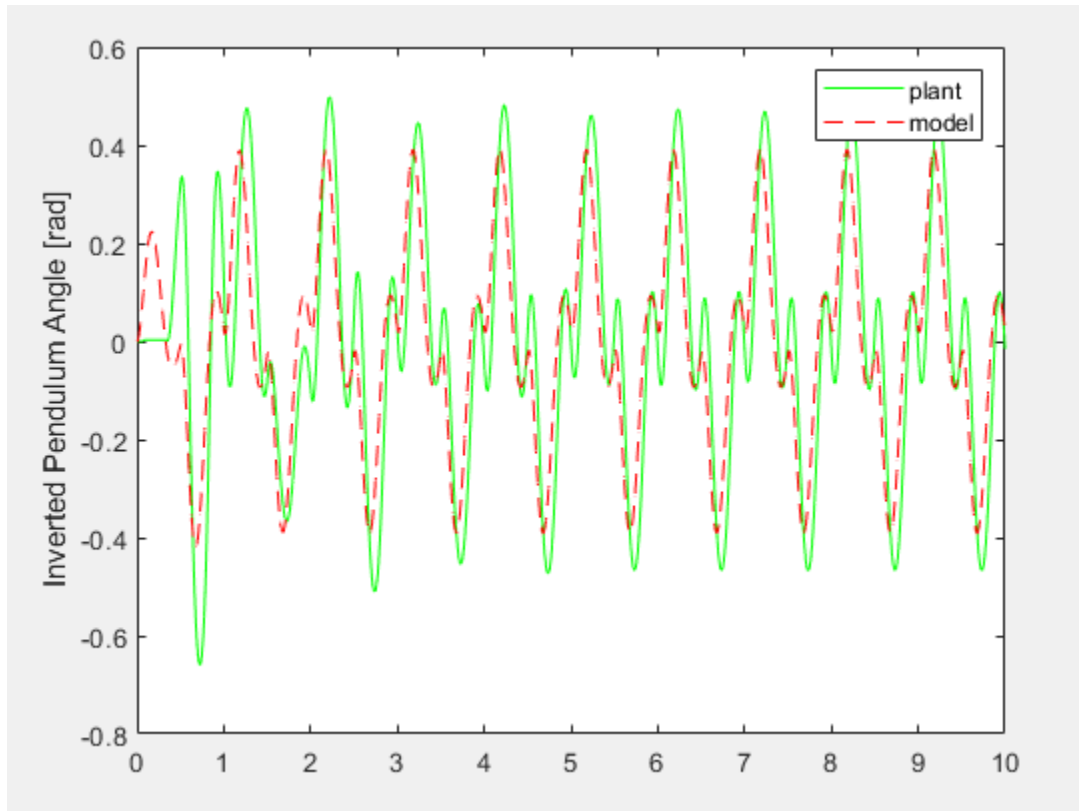
HIL-1 (qube_servo3_usb-0@tcpip://localhost:18923)

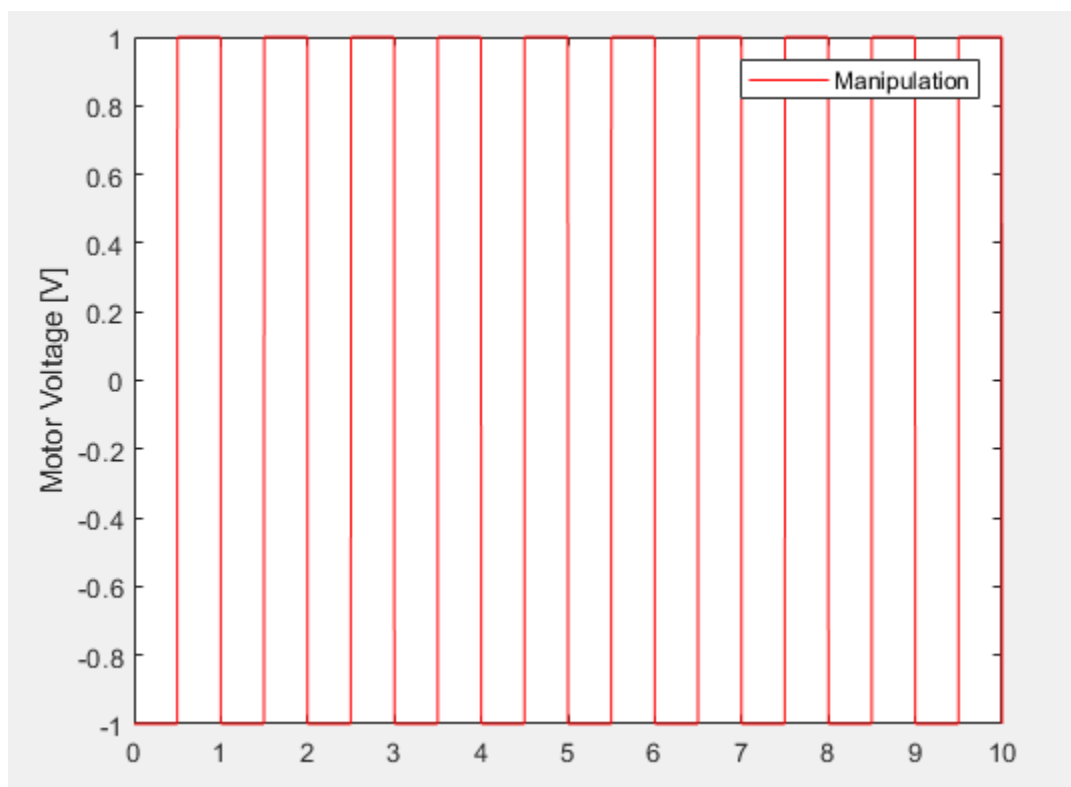
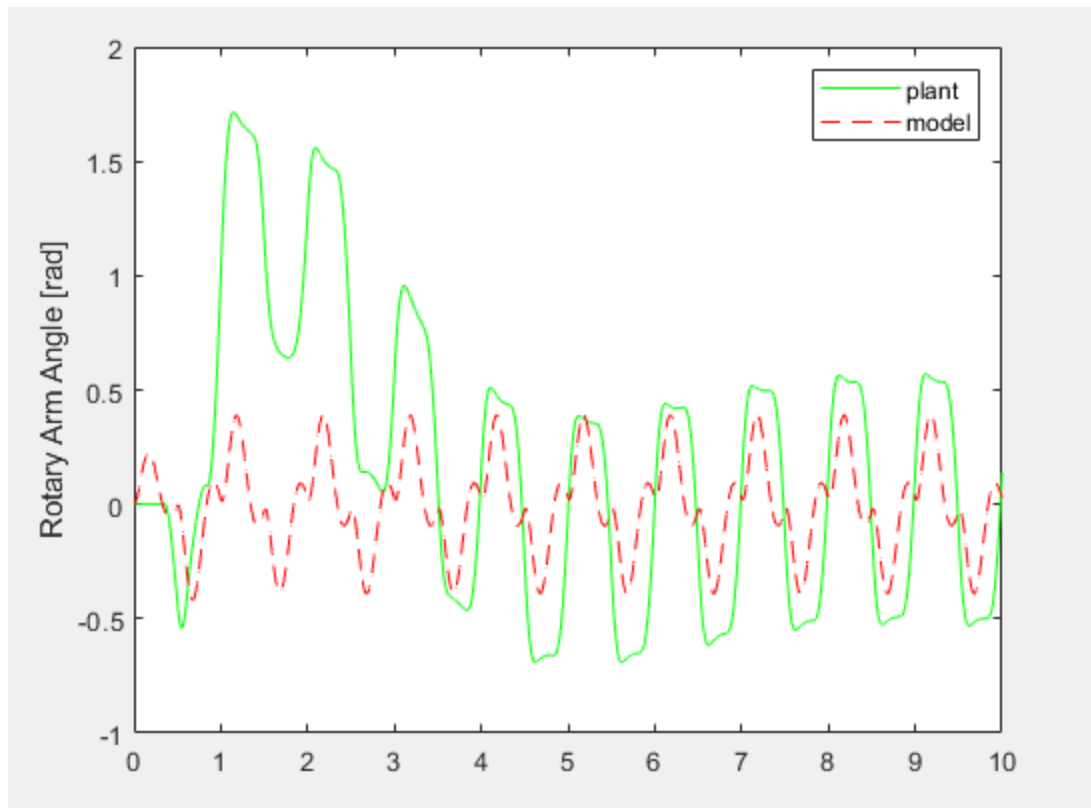


Figures



4. Linear State Response of the Rotary Pendulum System for the Virtual Lab and the Model (Matlab)





Matlab Script to plot figures

```

%% Step 3: Plot the response
load("a00834778_P9A.mat");
t = data(1,:);

voltage = data(2,:);

pos_arm = data(3,:);
pos_arm_model = data(4,:);
pos_pend = data(5,:);
pos_pend_model = data(6,:);

figure(1)
plot(t,voltage,"r");
legend("plant","model")
legend("Manipulation")
ylabel("Motor Voltage [V]");

figure(2)
plot(t,pos_arm,"g");
hold on
plot(t,pos_pend_model,"--r");
legend("plant","model")
ylabel("Rotary Arm Angle [rad]");

figure(3)
plot(t,pos_pend,"g");
hold on
plot(t,pos_pend_model,"--r");
legend("plant","model")
ylabel("Inverted Pendulum Angle [rad]");

```

5. Conclusion

In this practice, I successfully modeled the state-space representation of the rotary inverted pendulum (Furuta pendulum) system using first principles and validated it through both simulation and real-time experimentation (digital twin). By deriving and linearizing the system's equations of motion, I computed the state space representation matrices and evaluated the system's performance in open loop.

The results demonstrated that the derived model closely approximates the actual pendulum system under controlled conditions. Any discrepancies observed between the model and the

real system can likely be attributed to factors such as friction, unmodeled nonlinearities, or sensor inaccuracies.

Overall, the practice successfully highlighted the principles of state space representation, reinforcing its applicability in advanced control scenarios and the importance of thorough testing and validation in practical implementations.